

Upstox API: Beginner's Guide to Algorithmic Trading

Upstox offers a comprehensive REST API for building trading platforms and algo trading systems. The official **upstox-python** SDK wraps all Upstox API endpoints, enabling Python programs to fetch market data, place orders, and manage portfolios. This guide walks through every endpoint of the Upstox Python SDK (API v2), focusing on practical algorithmic trading use cases. It includes setup instructions, authentication steps, and clear examples for key functions like fetching quotes, placing orders, and tracking positions. Wherever possible, we link to official documentation to ensure accuracy ¹ ².

Setup and Authentication

1. **Install the SDK.** Use pip to install the Upstox Python client:

```
pip install upstox-python-sdk
```

Then import it in your code:

```
import upstox_client
```

(Source: official Upstox SDK docs ¹.)

2. **Configure the client.** Create a `Configuration` object. For testing, enable sandbox mode (`sandbox=True`); otherwise, leave it `False` for live trading. Assign your access token to `configuration.access_token`. For example:

```
config = upstox_client.Configuration(sandbox=True)
config.access_token = 'YOUR_ACCESS_TOKEN'
api_client = upstox_client.ApiClient(config)
```

This tells the SDK to use your credentials and (optionally) the sandbox endpoint ¹.

3. **Obtain an access token.** Upstox uses OAuth 2.0 for user authentication ². All logins happen via Upstox's web portal, which returns an auth code. You then exchange this code for an access token by calling the `LoginApi`. For example:

```
login_api = upstox_client.LoginApi(api_client)
token_response = login_api.token(body={"code": "<AUTH_CODE>", "client_id":
```

```
"<API_KEY>", "client_secret": "<API_SECRET>"}))
access_token = token_response.access_token
```

(Refer to Upstox's [Authentication docs] for detailed flow ²). Set `config.access_token = access_token` after retrieving it.

4. **Sandbox vs Live.** Upstox provides a sandbox environment for safe testing. In sandbox mode (`sandbox=True`), use a sandbox access token. The SDK examples below use sandbox mode by default for illustration ¹ .

5. **Handle exceptions.** All API calls may raise `ApiException` on error. Use try/except around API calls to catch and print errors. For example:

```
try:
    response = api_instance.get_order_book()
except upstox_client.rest.ApiException as e:
    print("API Error: %s" % e)
```

Citation: Upstox's sample code shows configuring OAuth and handling exceptions ³ ⁴ .

Key Endpoints Overview

Below are the primary Upstox API categories and endpoints, with descriptions, parameters, and code examples.

- **Market Data:** Retrieve live quotes and historical price data.
- **Orders:** Place, modify, or cancel orders; check order status.
- **Portfolio:** View holdings, positions, and account margin/funds.
- **Trade Reports:** Fetch trade history and profit/loss reports.
- **User Info:** Get user profile and fund details.
- **Utilities:** Exchange timings and holidays.
- **WebSocket Streams:** Subscribe to real-time market and portfolio updates.

The official SDK docs list all endpoints (see table at the top of this guide) ⁵ . Each endpoint below is explained with its parameters and return data.

Market Data Access

Live Market Quotes

- `get_full_market_quote` (`MarketQuoteApi`) – Get full quote details (bid/ask, LTP, etc.) for one or more instruments.
- `get_market_quote_ohlc` – Get the OHLC (open-high-low-close) data for instruments.
- `ltp` – Get the latest traded price (LTP) and basic quote for instruments.

Parameters: Typically a list of instrument tokens (e.g. "NSE_EQ|RELIANCE") and optional `api_version`.

Example: Get LTP for Reliance and TCS:

```
from upstox_client import MarketQuoteApi
api = MarketQuoteApi(api_client)
instruments = ["NSE_EQ|RELIANCE", "NSE_EQ|TCS"]
response = api.ltp(instruments)
print("LTP Data:", response)
```

This returns a dictionary mapping each token to its LTP and timestamp. (Use `get_full_market_quote()` for more detail.)

Source: Endpoint documented in SDK list [6](#).

Historical Candle Data

- `get_historical_candle_data` (`HistoryApi`) - Retrieve historical price candles for an instrument up to a date (e.g. daily, 1min).
- `get_historical_candle_data1` - Same as above but specify a date range (from and to).
- `get_intra_day_candle_data` - Same-day intraday candles with specified interval.

Usage: Specify instrument token, interval (like `1minute`, `day`), and date(s). The API returns time-series OHLC data.

Example: Get 1-minute candles for one day:

```
from upstox_client import HistoryApi
import datetime

history = HistoryApi(api_client)
instrument = "NSE_EQ|RELIANCE"
today = datetime.date.today()
yesterday = today - datetime.timedelta(days=1)
candles = history.get_historical_candle_data1(instrument, interval="1minute",
                                              to_date=today.isoformat(),
                                              from_date=yesterday.isoformat())
print(f"Candle data from {yesterday} to {today}:", candles)
```

This fetches 1-minute historical candles for Reliance from yesterday to today.

Note: Upstox Developer docs provide REST examples for historical data retrieval [7](#).

Option Chain Data

- `get_option_contracts` (`OptionsApi`) - Fetch available option contracts (strike prices) for an underlying.

- `get_put_call_option_chain` – Get the entire option chain (prices for call and put options) for a symbol.

Example: Retrieve option contracts for a stock:

```
from upstox_client import OptionsApi
options_api = OptionsApi(api_client)
symbol = "NSE_EQ|RELIANCE"
contracts = options_api.get_option_contracts(exchange_segment="NSE-OPT",
symbol=symbol)
print("Available strikes:", contracts)
```

Then get the option chain:

```
chain = options_api.get_put_call_option_chain(exchange_segment="NSE-OPT",
symbol=symbol)
print("Option Chain:", chain)
```

This helps algorithms gauge volatility or volatility strategies.

Market Info (Timings & Holidays)

- `get_market_status` (`MarketHolidaysAndTimingsApi`) – Check if a given exchange is open or closed on a date.
- `get_exchange_timings` – Get opening/closing times for a date.
- `get_holidays` – List all holidays for the year.
- `get_holiday` – Check if a specific date is a holiday.

These endpoints are useful for scheduling algorithms (e.g., skip trading on holidays). For example:

```
from upstox_client import MarketHolidaysAndTimingsApi
market_api = MarketHolidaysAndTimingsApi(api_client)
status = market_api.get_market_status("NSE")
print("NSE Market status:", status)
```

Order Placement and Management

Placing Orders

- `place_order` (`OrderApi` or `OrderApiV3`) – Place a single new order. Specify instrument token, quantity, price, type (LIMIT/MARKET), product (MIS/CNC), buy/sell, etc.
- `place_multi_order` – Send multiple orders in a single request.
- `place_order` (`OrderApiV3`) – New version for placing orders (similar parameters in a request body).

Example (single order): Place a limit buy order:

```
from upstox_client import OrderApi

order_api = OrderApi(api_client)
request = upstox_client.PlaceOrderRequest(
    instrument_token="NSE_EQ|RELIANCE",
    quantity=1,
    order_type="LIMIT",
    transaction_type="BUY",
    price=2500.00,
    validity="DAY",
    product="MIS",
    trigger_price=0.0
)
response = order_api.place_order(body=request)
print("Placed order:", response)
```

This submits a market BUY order for 1 share of Reliance. Replace `PlaceOrderRequest` with `PlaceOrderV3Request` when using `OrderApiV3`. The SDK docs example shows a similar code snippet for placing an order ⁸.

Modifying and Canceling Orders

- `modify_order` (`OrderApi` or `OrderApiV3`) – Change parameters (e.g. price, quantity) of an existing order. Provide the order ID or exchange ID.
- `cancel_order` – Cancel an existing order. You typically supply `order_id` or `client_order_id`.
- `cancel_multi_order` – Cancel multiple orders at once by IDs.

Example (cancel): Cancel an order by its ID:

```
order_api = OrderApi(api_client)
order_id = "123456789"
order_api.cancel_order(order_id=order_id)
print("Order cancelled:", order_id)
```

Always check the returned status or catch exceptions to confirm cancellation.

Order Book and Status

- `get_order_book` – Retrieve all open (day) orders for the user.
- `get_order_status` – Check status of a specific order by ID.
- `get_order_details` / `get_order_history` – Get historical orders (all days or specific day).

- `get_trade_history` (`get_trades_by_date_range`) – Fetch trade executions (filled orders) over a date range.
- `get_trades_by_order` – Get trades for a specific order ID.

Example: List current open orders:

```
order_book = order_api.get_order_book()
for order in order_book:
    print(order)
```

This shows all day's orders, including their IDs. You can then call `get_trade_history()` or `get_trades_by_order(order_id)` to see executed trades. These endpoints help algorithms verify execution and manage partial fills.

Special Order Actions

- `exit_positions` – Exit (sell/cover) all open positions immediately. Useful for stop-loss or closing at end-of-day.
- `convert_positions` – Convert positions between intraday (MIS) and delivery (CNC) products.
- `exit_positions` **example:**

```
response = order_api.exit_positions()
print("Exit all positions response:", response)
```

- `place_multi_order` example:

```
multi_request = upstox_client.MultiOrderRequest(orders=[ ... ]) # list of PlaceOrderRequest
multi_response = order_api.place_multi_order(body=multi_request)
print("Multi-order response:", multi_response)
```

Note: The newer `OrderApiV3` provides similar methods (`place_order`, `modify_order`, `cancel_order` under `/v3/order/...` paths) and uses different request models (`PlaceOrderV3Request`). Use whichever version your platform supports; v3 may have slight enhancements.

Portfolio and Account Management

Positions and Holdings

- `get_positions` (`PortfolioApi`) – Fetch all open positions (intraday).
- `get_holdings` – Fetch long-term holdings (delivery positions).
- `convert_positions` – Convert positions between trading segments (e.g. MIS to CNC).

Example: Check open positions:

```

from upstox_client import PortfolioApi

portfolio_api = PortfolioApi(api_client)
positions = portfolio_api.get_positions()
print("Open positions:", positions)

```

Holdings:

```

holdings = portfolio_api.get_holdings()
print("Current holdings:", holdings)

```

Funds and Margins

- `get_user_fund_margin` (`UserApi`) - Retrieve available funds and margin limits.
- `get_profile` - Get basic user profile (name, ID, etc.).

Example: Check funds:

```

from upstox_client import UserApi

user_api = UserApi(api_client)
funds = user_api.get_user_fund_margin()
print("Funds and margin:", funds)

```

This returns cash balances and usable margin, essential for position sizing in algos.

Trade Profit & Loss

- `get_profit_and_loss_charges` (`TradeProfitAndLossApi`) - Get total profit/loss and charges on trades.
- `get_trade_wise_profit_and_loss_data` - Detailed P&L per trade.
- `get_trade_wise_profit_and_loss_meta_data` - Metadata for P&L fields.

Use these to analyze strategy performance. For example:

```

pnl_api = upstox_client.TradeProfitAndLossApi(api_client)
pnl_report = pnl_api.get_trade_wise_profit_and_loss_data()
print("Trade P&L report:", pnl_report)

```

Trade History and Charges

- `get_trades_by_date_range` (`PostTradeApi`) – Get all historical trades (executions) between dates.
- `get_brokerage` and `post_margin` (`ChargeApi`) – Calculate brokerage and margin requirements for hypothetical trades.

Example (historical trades):

```
from datetime import date
post_trade = upstox_client.PostTradeApi(api_client)
trades = post_trade.get_trades_by_date_range(from_date=date(2023,1,1),
to_date=date.today())
print("Historical trades:", trades)
```

The `get_brokerage` endpoint can simulate charges:

```
charge_api = upstox_client.ChargeApi(api_client)
brokerage = charge_api.get_brokerage(
    instrument_token="NSE_EQ|RELIANCE", quantity=10, product="MIS",
    transaction_type="BUY", price=2500.0, api_version="2"
)
print("Estimated brokerage:", brokerage)
```

(Example usage of `get_brokerage` and `post_margin` is shown in Upstox docs [9](#) [10](#).)

WebSocket Streaming

Upstox provides WebSocket feeds for real-time data: - **MarketDataStreamerV3** – Connect to the market data WebSocket and subscribe to live updates for instruments.

- **PortfolioDataStreamer** – Connect to receive live order/portfolio updates (order fills, position changes).

MarketDataStreamerV3 Example:

```
import upstox_client

def on_message(msg):
    print("Market update:", msg)

config = upstox_client.Configuration()
config.access_token = access_token # set your access token
stream = upstox_client.MarketDataStreamerV3(
    upstox_client.ApiClient(config),
    instrument_keys=["NSE_INDEX|Nifty 50", "NSE_INDEX|Nifty Bank"],
```



```

        mode="full"
    )
    stream.on("message", on_message)
    stream.connect()

```

This connects to the market feed and prints each update received ¹¹. Use `streamer.subscribe(...)` in `on_open()` to add subscriptions dynamically.

PortfolioDataStreamer Example:

```

import upstox_client

def on_portfolio_msg(msg):
    print("Portfolio update:", msg)

config = upstox_client.Configuration()
config.access_token = access_token
p_stream = upstox_client.PortfolioDataStreamer(
    upstox_client.ApiClient(config),
    order_update=True, position_update=True, holding_update=True,
    gtt_update=False
)
p_stream.on("message", on_portfolio_msg)
p_stream.connect()

```

This stream notifies on order and position updates. Both streamers run asynchronously and invoke the callback on new messages ¹².

Use streaming for high-frequency or event-driven strategies. For example, an algo might subscribe to Nifty index updates to trigger signals.

Tips & Best Practices for Beginners

- **Start in Sandbox.** Always test your strategies in sandbox mode first ¹. Use fake tokens and verify all logic before going live.
- **Mind the Rate Limits.** Upstox enforces strict rate limits (e.g. 50 requests/sec, 500/min for standard APIs) ¹³. Batch requests, use multi-order endpoints, or slow down loops to stay within limits.
- **Use WebSockets.** For real-time strategies, use `MarketDataStreamerV3` instead of polling REST APIs. It's more efficient and avoids hitting rate limits.
- **Modularize Code.** Write helper functions for common tasks (e.g. fetching LTP, placing an order, checking fills). This keeps your algo code clean.
- **Error Handling.** Catch `ApiException` around API calls and implement retries or logging. Network or API errors can occur, so ensure your algorithm can handle them.
- **Logging and Monitoring.** Log every order action and API response. In live trading, log to a file or monitoring dashboard to debug issues.

- **Trade within hours.** Use market timing APIs to ensure your algorithms only trade when exchanges are open. For example, check `get_market_status()` before placing day orders.
- **Example Workflow:** A simple intraday strategy flow might be:
 - At market open, fetch list of instrument tokens for stocks you trade.
 - Use the WebSocket to get live LTP or OHLC bars.
 - When your entry conditions are met, call `place_order()`.
 - Periodically check `get_order_status()` or listen via the portfolio stream for fills.
 - After a trade, update positions and possibly set exit orders (stop-loss/take-profit).
 - At end of day, call `exit_positions()` to close any leftover intraday trades.
- **Learn the API.** Refer to the official Upstox API docs and this SDK documentation regularly. All endpoints, parameters, and example code are documented (see [Upstox API Developer Documentation](#) and the SDK's GitHub pages ⁵ ⁹).

By following these practices and using the Upstox SDK methods above, beginners can confidently build and test simple algorithmic trading strategies.

References: Official Upstox API docs and SDK examples are cited throughout (e.g. OAuth flow ² , endpoint summaries ⁵ , sandbox example ¹ , and sample API calls ⁹ ¹³). For more details, visit the [Upstox Developer API Documentation](#).

¹ ⁵ ¹² Upstox Python SDK for API v2 | upstox-python
<https://upstox.github.io/upstox-python/>

² Authentication | Upstox Developer API
<https://upstox.com/developer/api-documentation/authentication/>

³ ⁴ ⁹ ¹⁰ upstox_client.ChargeApi | upstox-python
<https://upstox.github.io/upstox-python/docs/ChargeApi.html>

⁶ ⁸ ¹¹ GitHub - upstox/upstox-python: Official Python SDK for accessing Upstox API
<https://github.com/upstox/upstox-python>

⁷ Historical Candle Data | Upstox Developer API
<https://upstox.com/developer/api-documentation/example-code/historical-data/historical-candle-data/>

¹³ Rate Limits | Upstox Developer API
<https://upstox.com/developer/api-documentation/rate-limiting/>